

B.V.V.SANGHA'S BASAVESHWAR ENGINEERING COLLEGE, BAGALKOT

DEPARTMENT PUTER SCIENCE AND ENGINEERING

Cyber Security Report

Name: Jayshree. kumbar

USN: 2BA24CS046

Roll No : 40

Project Title: Data security and Encryption

Semester: II

Division: I

Submitted to: Dr.G.B.Chittapur

Group Members:

S.No.	Name.	Roll No.	USN
1.	Jayshree Kumbar	40.	2BA24CS046
2.	Bhagya Akki.	23.	2BA24CS028
3.	Kavita Rathod	46.	2BA24CS055

INTRODUCTION



(Data Security and File Encryption Using Python (Fernet Algorithm))

➤ Introduction to Data Security

Data Security is the practice of protecting digital information from unauthorized access or corruption.

In today's world, protecting files and user data is more important than ever.

Encryption ensures that data remains confidential and secure, even if accessed by unauthorized users.

Python offers simple yet powerful libraries to implement encryption.

➤ What is Encryption?

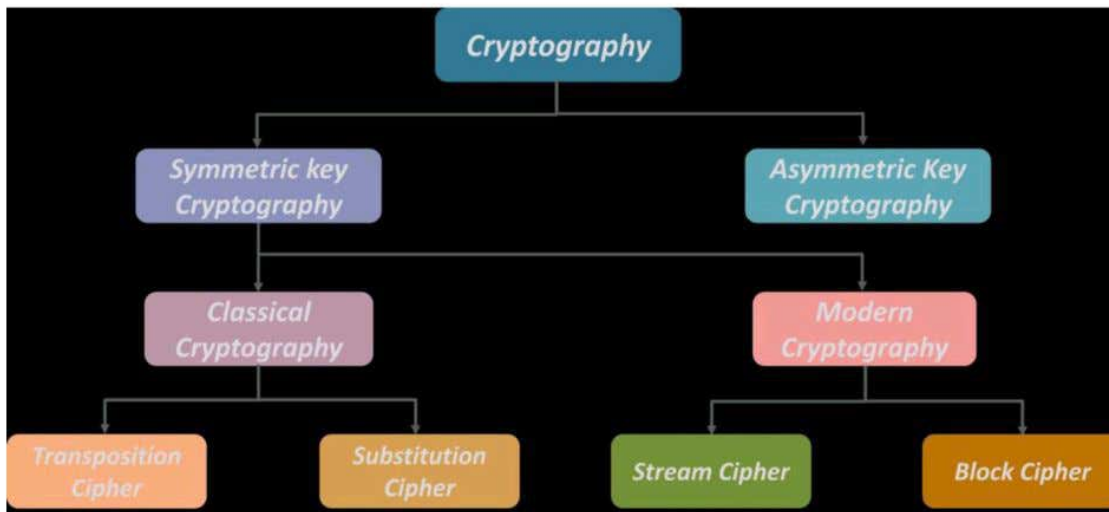


Encryption is the process of converting plain text into unreadable cipher text using a secret key.

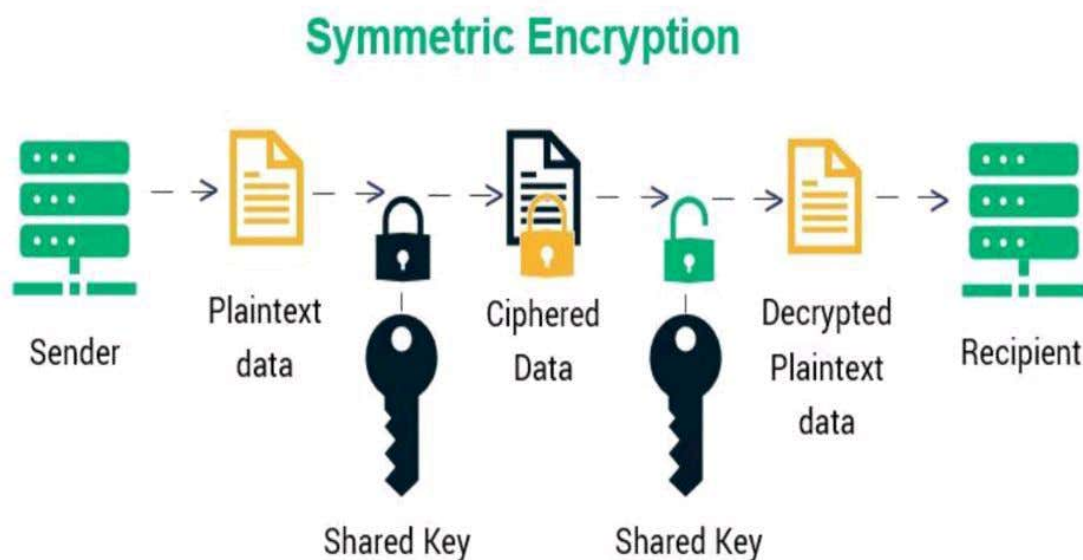
Only those who have the correct decryption key can convert cipher text back to plain text.

➤ Types of Encryption:

Symmetric Encryption (same key for encryption & decryption)



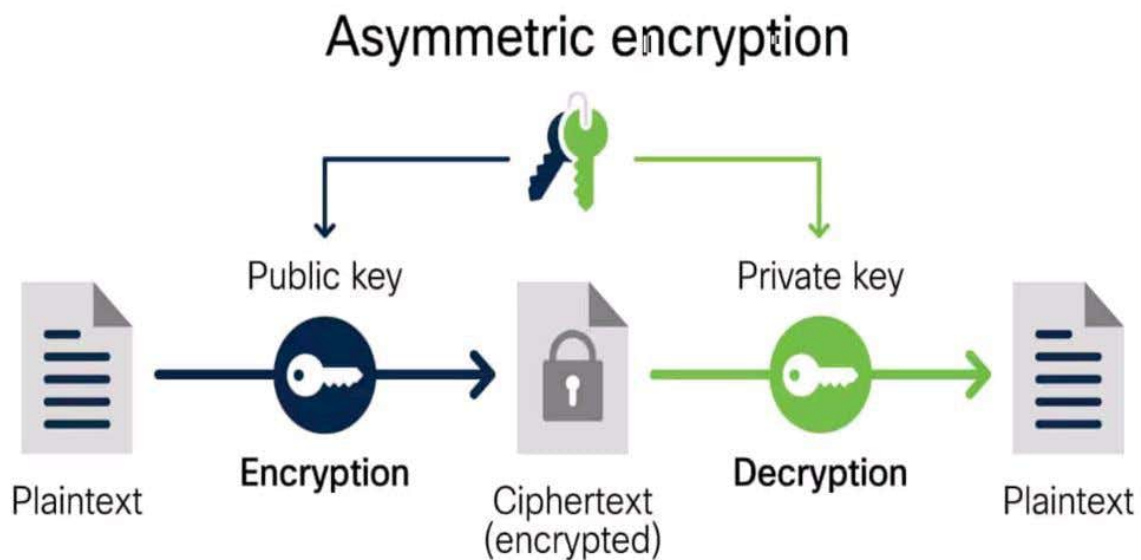
▪ SYMMETRIC KEY CRYPTOGRAPHY



▪ ASYMMETRIC KEY CRYPTOGRAPHY

Asymmetric Encryption (public/private keys)

We use Symmetric Encryption (Fernet) in this project.



About the Fernet Algorithm

Fernet is a symmetric encryption method provided by Python's cryptography module.

- **Ensures:**

Data confidentiality

Integrity (checks if data is tampered)

Authenticity

Uses 128-bit AES in CBC mode with PKCS7 padding and HMAC.

➤ **Project Objective**

- Develop a Python application that:
 - Generates an encryption key.
 - Encrypts any given file using that key.
 - Decrypts the encrypted file using the same key.
- Ensure simple command-line interaction.

- Demonstrate the basic working of real-world encryption.

➤ **Project Workflow**

Step 1: Generate and store secret key

Step 2: Encrypt selected file using the key

Step 3: Save encrypted file as .enc

Step 4: Decrypt the .enc file using the key

Step 5: Restore the original file as *_decrypted

➤ **Main Functions:**

➤ generate_key()

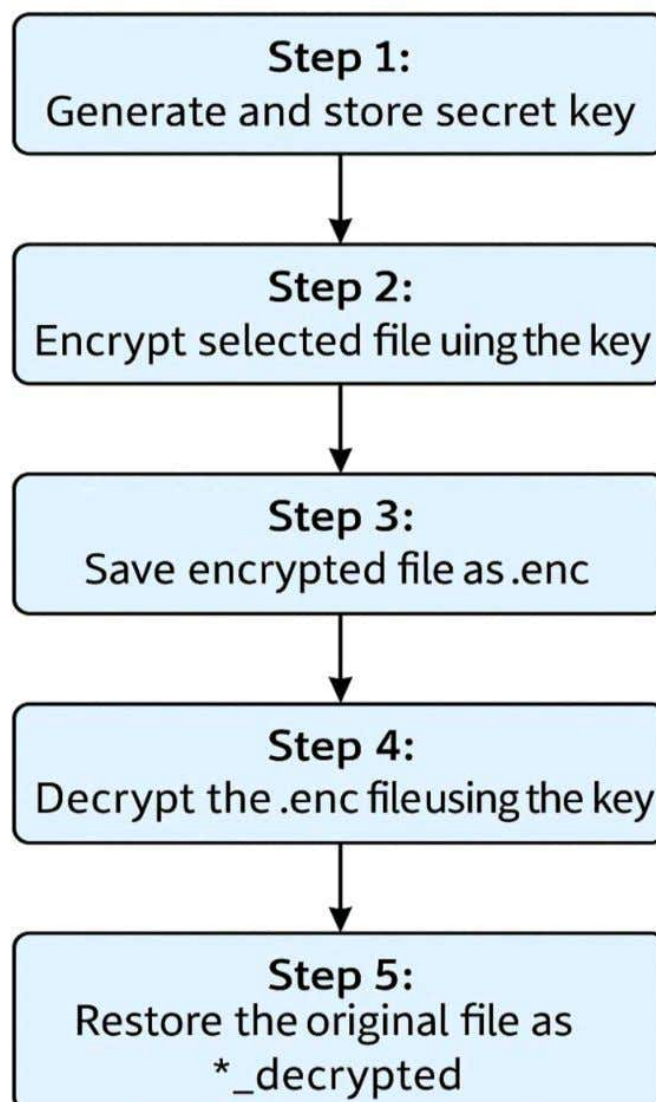
➤ load_key()

➤ encrypt_file(filename, key)

➤ decrypt_file(encrypted_filename, key)

➤ Python Code Structure

Project Workflow



➤ **Execution Flow:**

User chooses operation (gen, enc, dec)

Based on input, appropriate function is called

➤ **Sample Code Snippet**

```
from cryptography.fernet import Fernet  
  
# Generate Key  
def generate_key():  
    key = Fernet.generate_key  
    with open("secret.key", "wb") as key_file:  
        key_file.write(key)  
  
# Encrypt File  
def encrypt_file(filename, key):  
    f = Fernet(key)
```

```
with open(filename, "rb") as file:  
    data = file.read()  
encrypted_data = f.encrypt(data)  
with open(filename + ".enc", "wb") as file:  
    file.write(encrypted_data)
```

➤ **User Interface (Command Line)**

User Input Options:

'gen' – Generate encryption key

'enc' – Encrypt file

'dec' – Decrypt file

➤ **Example Flow:**

Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to decrypt:

> enc

Enter filename to encrypt: mydoc.txt

Sample Output Screenshots

Here are some screenshots showing:

- Key generation
- Encryption success
- Decryption success
- File before & after encryption

This helps visualize how our project works in real time

Usage

Help Variable Explorer Plots Files

Console 1/A X

```
TypeError: 'int' object is not iterable

In [11]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to
decrypt: gen
✅ Key generated and saved to 'secret.key'.

In [12]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to
decrypt: enc
Enter the name of the file to encrypt: exp.txt
🔒 File 'exp.txt' encrypted successfully as 'exp.txt.enc'.

In [13]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to
decrypt: dec
Enter the name of the encrypted file to decrypt (with .enc):
exp.txt.enc
🔒 File 'exp.txt.enc' decrypted successfully as
'exp.txt_decrypted'.

In [14]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
```

Python Console History

conda (Python 3.11.7) Completions: conda LSP: Python Line 58, Col 99 UTF-8 CRLF RW Mem 79%

6:45 AM 6/12/2025



Working of this code

```
pyder (Python 3.11)
Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\91935\OneDrive\Desktop\pro.py

temp.py X l.py X untitled0.py X untitled1.py X pro.py X secret.key X exp.txt X exp.txt.enc X exp.txt_de

"""
Created on Thu Jun 12 06:13:22 2025

@author: 91935
"""

from cryptography.fernet import Fernet, InvalidToken
import os

# Generate and save a key
def generate_key():
    key = Fernet.generate_key()
    with open("secret.key", "wb") as key_file:
        key_file.write(key)
    print("✔ Key generated and saved to 'secret.key'.")

# Load the existing key
def load_key():
    try:
        return open("secret.key", "rb").read()
    except FileNotFoundError:
        print("✖ Error: 'secret.key' not found. Generate the key first.")
        return None

# Encrypt a file
def encrypt_file(filename, key):
    if not os.path.exists(filename):
        print("✖ Error: File not found.")
        return

    try:
        f = Fernet(key)
        with open(filename, "rb") as file:
            data = file.read()
        encrypted_data = f.encrypt(data)
        enc_filename = filename + ".enc"
        with open(enc_filename, "wb") as file:
            file.write(encrypted_data)
        print(f"✔ File '{filename}' encrypted successfully as '{enc_filename}'.")
    except InvalidToken:
        print("✖ Invalid key provided.")
```

```

temp.py x 1.py x undecrypt.py x undecrypt.py x pro.py x secretkey x exploit x exploitenc x exploit_decrypted x
47     print("❌ Error: Encrypted file not found.")
48     return
49
50 try:
51     f = Fernet(key)
52     with open(encrypted_filename, "rb") as file:
53         encrypted_data = file.read()
54         decrypted_data = f.decrypt(encrypted_data)
55         decrypted_filename = encrypted_filename.replace(".enc", "_decrypted")
56         with open(decrypted_filename, "wb") as file:
57             file.write(decrypted_data)
58     print(f"📁 File '{encrypted_filename}' decrypted successfully as '{decrypted_filename}'.")
59 except InvalidToken:
60     print("❌ Decryption failed: Invalid key or corrupted file.")
61 except Exception as e:
62     print(f"❌ Decryption error: {e}")
63
64 # Main menu
65 if __name__ == "__main__":
66     print("🔧 FILE ENCRYPTION TOOL")
67     choice = input("Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to decrypt: ").strip().lower()
68
69     if choice == "gen":
70         generate_key()
71
72     elif choice == "enc":
73         filename = input("Enter the name of the file to encrypt: ").strip()
74         key = load_key()
75         if key:
76             encrypt_file(filename, key)
77
78     elif choice == "dec":
79         filename = input("Enter the name of the encrypted file to decrypt (with .enc): ").strip()
80         key = load_key()
81         if key:
82             decrypt_file(filename, key)
83
84     else:
85         print("❌ Invalid input. Please enter 'gen', 'enc', or 'dec'.")

```

```
temp.py X l.py X untitled0.py X untitled1.py X pro.py X secret.key X exp.bt X exp.bt.enc X exp.bt_decrypted X

    enc_filename = filename + ".enc"
    with open(enc_filename, "wb") as file:
        file.write(encrypted_data)
    print(f"🔒 File '{filename}' encrypted successfully as '{enc_filename}'.")
except Exception as e:
    print(f"❌ Encryption failed: {e}")

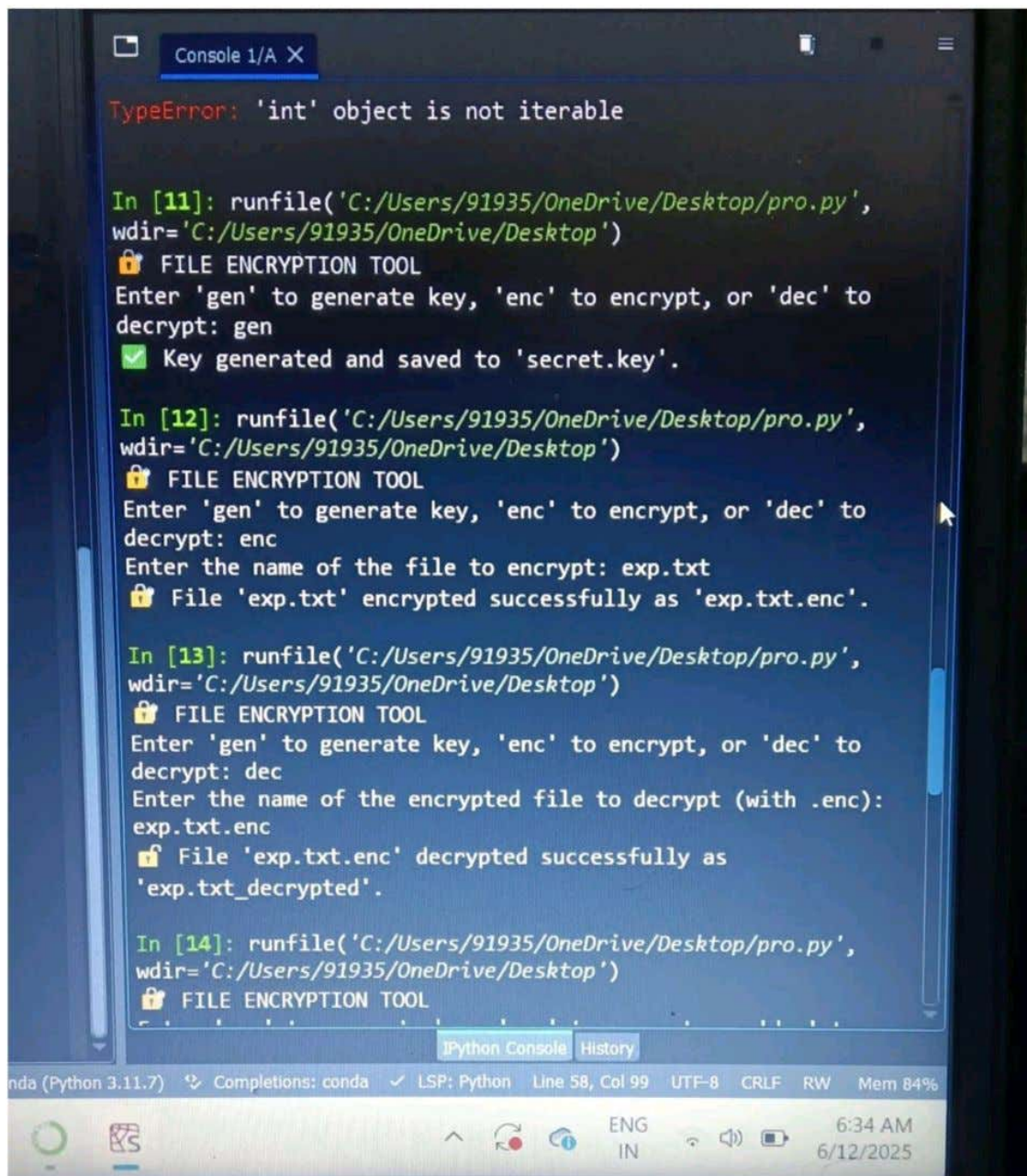
# Decrypt a file
def decrypt_file(encrypted_filename, key):
    if not os.path.exists(encrypted_filename):
        print("❌ Error: Encrypted file not found.")
        return
    try:
        f = Fernet(key)
        with open(encrypted_filename, "rb") as file:
            encrypted_data = file.read()
        decrypted_data = f.decrypt(encrypted_data)
        decrypted_filename = encrypted_filename.replace(".enc", "_decrypted")
        with open(decrypted_filename, "wb") as file:
            file.write(decrypted_data)
        print(f"🔓 File '{encrypted_filename}' decrypted successfully as '{decrypted_filename}'.")
    except InvalidToken:
        print("❌ Decryption failed: Invalid key or corrupted file.")
    except Exception as e:
        print(f"❌ Decryption error: {e}")

# Main menu
if __name__ == "__main__":
    print("🔐 FILE ENCRYPTION TOOL")
    choice = input("Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to decrypt: ").strip().lower()

    if choice == "gen":
        generate_key()

    elif choice == "enc":
        filename = input("Enter the name of the file to encrypt: ").strip()
        key = load_key()
        if key:
```

➤ Output of code(Console)



```
Console 1/A X

TypeError: 'int' object is not iterable

In [11]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to
decrypt: gen
✅ Key generated and saved to 'secret.key'.

In [12]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to
decrypt: enc
Enter the name of the file to encrypt: exp.txt
🔒 File 'exp.txt' encrypted successfully as 'exp.txt.enc'.

In [13]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
Enter 'gen' to generate key, 'enc' to encrypt, or 'dec' to
decrypt: dec
Enter the name of the encrypted file to decrypt (with .enc):
exp.txt.enc
🔒 File 'exp.txt.enc' decrypted successfully as
'exp.txt_decrypted'.

In [14]: runfile('C:/Users/91935/OneDrive/Desktop/pro.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 FILE ENCRYPTION TOOL
```

Python Console History

conda (Python 3.11.7) Completions: conda ✓ LSP: Python Line 58, Col 99 UTF-8 CRLF RW Mem 84%

6:34 AM 6/12/2025

➤ **Modified code**

What This Script Does
Automatically:

**1.Generates secret.key if not
already present.**

1.Creates message.txt file
with a sample message if not
present.

1.Loads the key from
secret.key.

1.Encrypts message.txt and saves it as message.txt.enc.

❖ Output Files:

- Message.txt (original readable file)
- Secret.key (binary encryption key)
- Message.txt.enc (encrypted file)

```
p.py X data security and encryption.py X encl.py X untitled0.py X untitled1.py X in.txt X

"""
Created on Mon Jun 16 00:16:48 2025

@author: 91935
"""

from cryptography.fernet import Fernet

# Function to generate a key
def generate_key():
    key = Fernet.generate_key()
    with open("secret.key", "wb") as key_file:
        key_file.write(key)
    return key

# Function to load the key
def load_key():
    with open("secret.key", "rb") as key_file:
        return key_file.read()

# Function to encrypt a file
def encrypt_file(filename, key):
    fernet = Fernet(key)
    with open(filename, "rb") as file:
        original_data = file.read()
    encrypted_data = fernet.encrypt(original_data)

    encrypted_filename = filename + ".enc"
    with open(encrypted_filename, "wb") as file:
        file.write(encrypted_data)

    print(f"🔒 Encrypted '{filename}' → '{encrypted_filename}'")
    return encrypted_filename

# Function to decrypt a file
def decrypt_file(encrypted_filename, key):
    fernet = Fernet(key)
    with open(encrypted_filename, "rb") as file:
        encrypted_data = file.read()
```

1 Upcoming Earnings

Search

```
935 OneDrive\Desktop\enc1.py
p.py X data security and encryption.py X enc1.py X untitled0.py X untitled1.py X ln.txt X

    encrypted_data = file.read()
    decrypted_data = fernet.decrypt(encrypted_data)

    decrypted_filename = encrypted_filename.replace(".enc", "_decrypted.txt")
    with open(decrypted_filename, "wb") as file:
        file.write(decrypted_data)

    print(f"🔓 Decrypted '{encrypted_filename}' → '{decrypted_filename}'")
    return decrypted_filename

# =====
# 🚀 MAIN FUNCTION STARTS HERE
# =====
if __name__ == "__main__":
    print("🔒 ENCRYPTION & DECRYPTION DEMO")

    # Step 1: Create a sample file
    filename = "sample.txt"
    with open(filename, "w", encoding="utf-8") as f:
        f.write("Hey bestie!! This is a secret message 🥰")
    print(f"📄 Created sample file: {filename}")

    # Step 2: Generate and save key
    key = generate_key()
    print("🔑 Key generated and saved.")

    # Step 3: Encrypt the file
    encrypted_file = encrypt_file(filename, key)

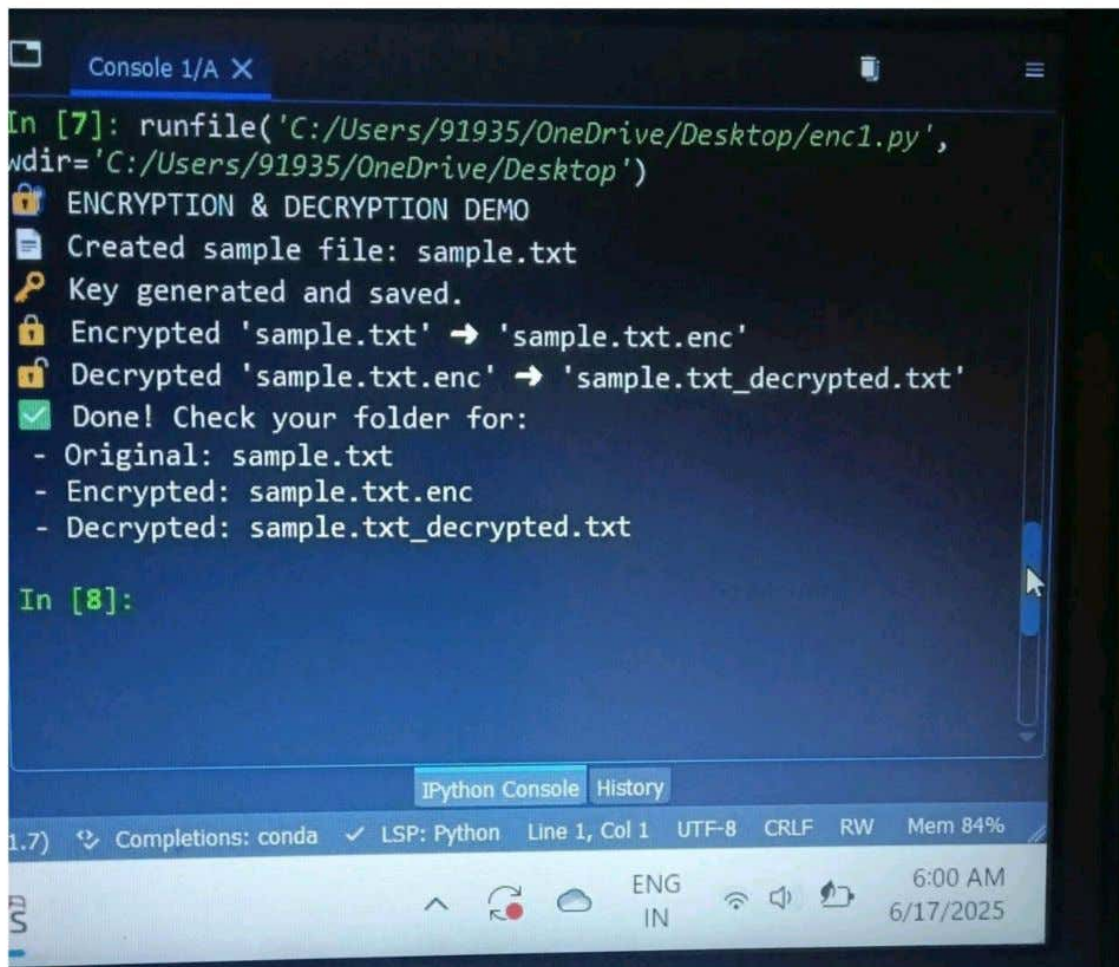
    # Step 4: Decrypt the file
    decrypted_file = decrypt_file(encrypted_file, key)

    # Done!
    print("✅ Done! Check your folder for:")
    print(f" - Original: {filename}")
    print(f" - Encrypted: {encrypted_file}")
    print(f" - Decrypted: {decrypted_file}")
```

trending videos
spaceballs 2 - O...

Search

➤ Output of code



```
Console 1/A X
In [7]: runfile('C:/Users/91935/OneDrive/Desktop/enc1.py',
wdir='C:/Users/91935/OneDrive/Desktop')
🔒 ENCRYPTION & DECRYPTION DEMO
📄 Created sample file: sample.txt
🔑 Key generated and saved.
🔒 Encrypted 'sample.txt' → 'sample.txt.enc'
🔓 Decrypted 'sample.txt.enc' → 'sample.txt_decrypted.txt'
✅ Done! Check your folder for:
- Original: sample.txt
- Encrypted: sample.txt.enc
- Decrypted: sample.txt_decrypted.txt

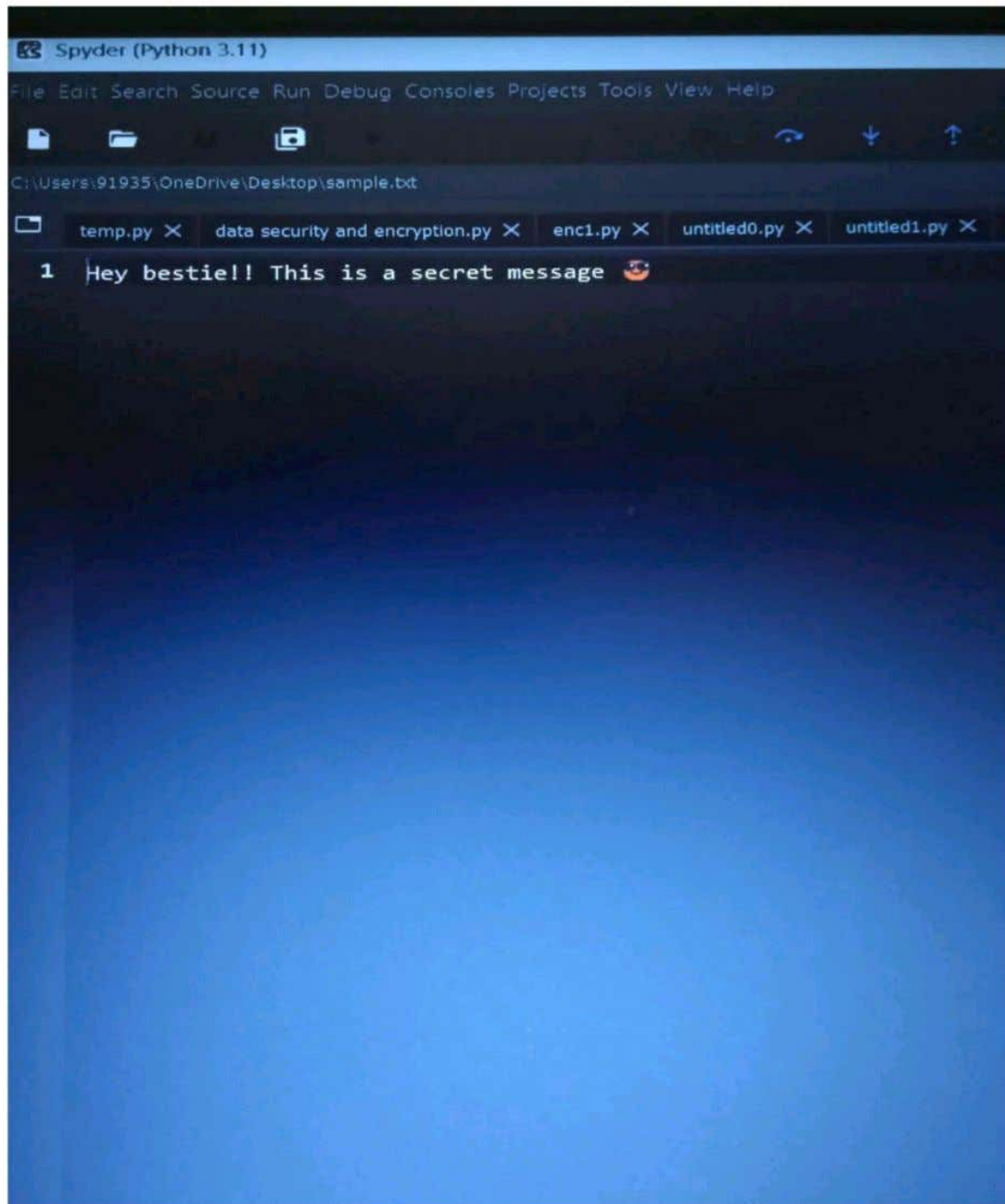
In [8]:
```

IPython Console History

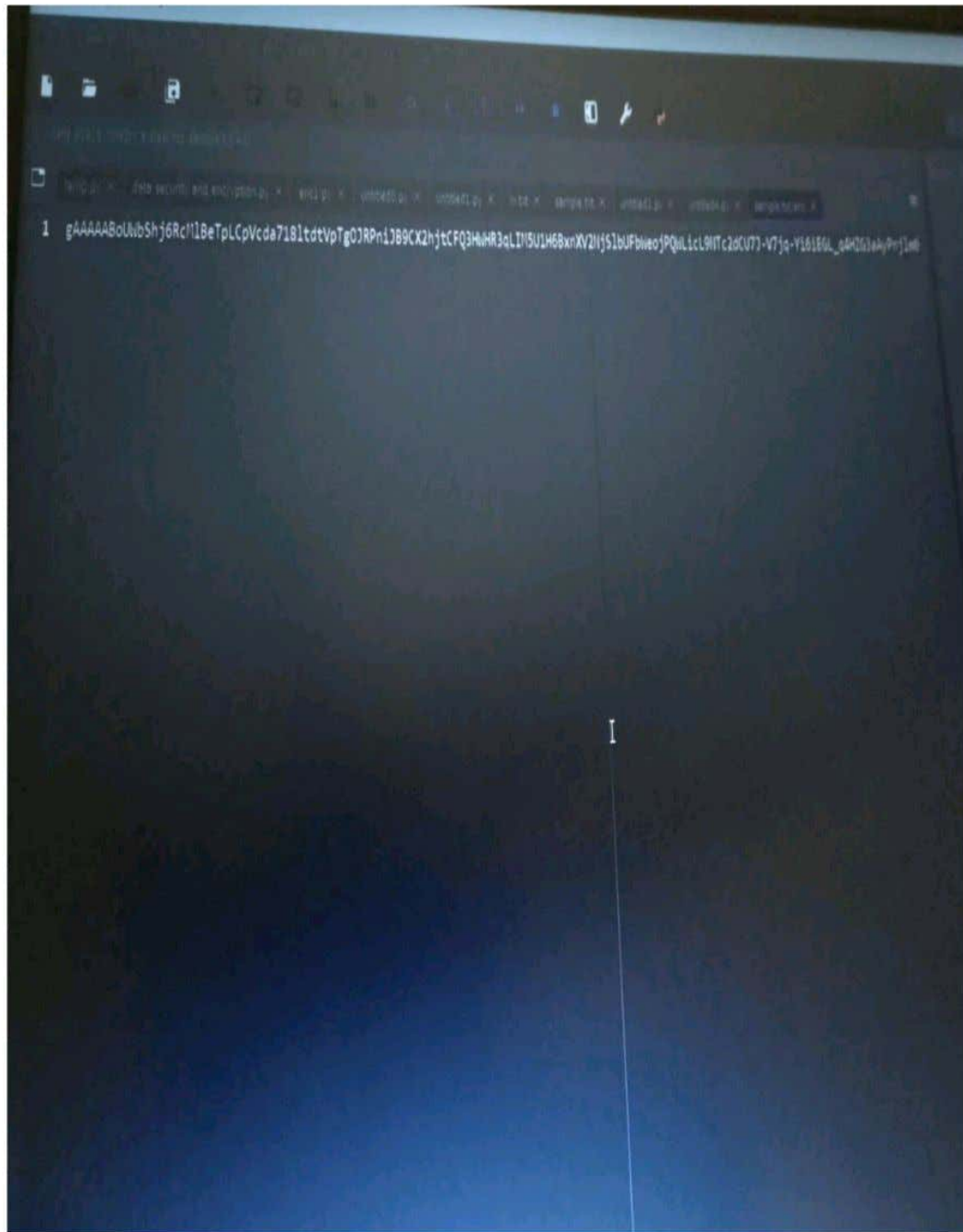
1.7) Completions: conda ✓ LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 84%

6:00 AM 6/17/2025

- File created

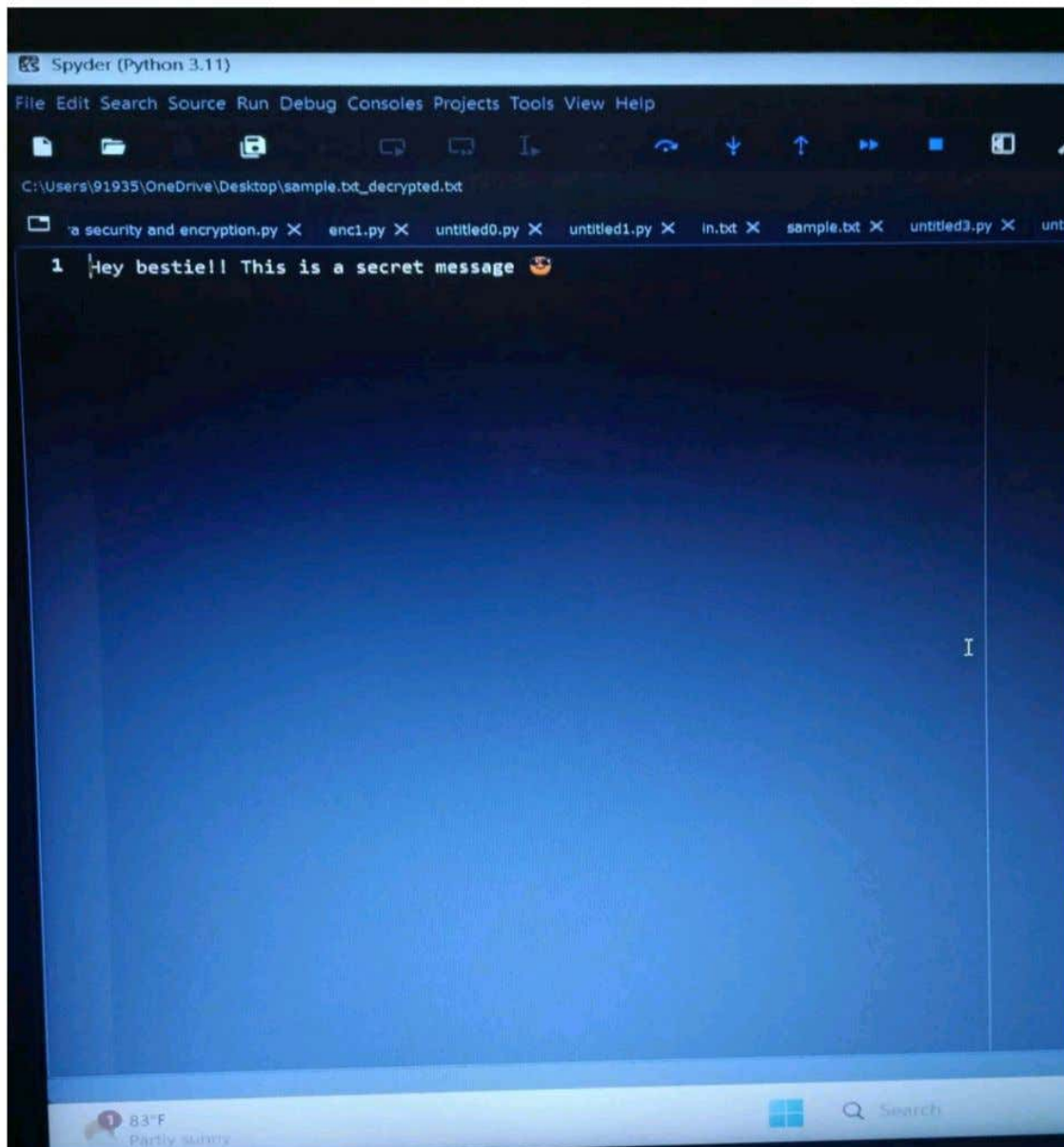


- File Encrypted successfully



This file is encrypted and is in unreadable form.

- File decrypted successfully



➤ File Encryption using Python (Fernet)

❖ This script helps you:

1. Generate a secure encryption key.
2. Encrypt any file using that key.

❖ Code Breakdown & Explanation:

1. Import Fernet:

```
from cryptography.fernet import Fernet
```

This imports the Fernet class from the cryptography library. Fernet handles secure encryption and decryption using symmetric keys.

2. Generate Key:

```
def generate_key(): key =  
Fernet.generate_key()          # Generate a  
new key with open("secret.key", "wb") as  
key_file: # Open file in write-binary mode  
key_file.write(key)             # Save the key to  
file
```

Explanation:

Fernet.generate_key() produces a secure 32-byte key.

The key is saved in a file named secret.key for reuse in encryption/decryption.

3. Encrypt File:

```
def encrypt_file(filename, key): f = Fernet(key)  
# Initialize Fernet with the provided key with  
open(filename, "rb") as file: data = file.read()  
# Read the file in binary mode  
encrypted_data = f.encrypt(data) # Encrypt
```

```
the file contents with open(filename + ".enc",  
"wb") as file: file.write(encrypted_data) #  
Save encrypted data as a new file
```

Explanation:

Opens the original file (e.g., data.txt).

Reads the data in binary.

Encrypts the data using the Fernet key.

Saves the result as data.txt.enc or similar.

Step-by-Step Demonstration

Let's walk through an example.

Step 1: Create a sample file

Create a text file named message.txt with the content: This is a secret message.

 Step 2: Run generate_key()

```
generate_key()
```

Result: A file named secret.key is created. It contains something like: b'0zFyN1c-7eVnTOtBSfsKmJH9kQGnXEZHZTRrci9r0Jg='

 Step 3: Load the Key

```
key = open("secret.key", "rb").read()
```

 Step 4: Encrypt the File

```
encrypt_file("message.txt", key)
```

Result: A new file named message.txt.enc is created. It contains encrypted binary data like:

gAAAAABk4Q1jXYnU5UJqypb...

This data is unreadable without the original key.

Output Files:

message.txt — original file (readable)

message.txt.enc — encrypted file
(unreadable)

secret.key — the encryption key (do not lose this!)

❖ What happens if the key is lost?

You will never be able to decrypt the encrypted file.

Always keep secret.key safe and secure.

❖ Summary

Function	Purpose
generate_key() stores a	Generates and secure KEY.
encrypt_file(filename) file using	Encrypts the given Fernet and saves a new .enc Files.

➤ Security Consideration

- Never share your secret.key publicly
- Store the key securely (e.g., encrypted USB or environment variable)
- Fernet ensures encryption cannot be reversed without the key
- Protect against key theft and misuse

➤ Applications

- File storage security (e.g., cloud backups)
- Securing sensitive documents
- Data transfer in secure communication
- Personal projects or internal tools

➤ **Advantages of Using Fernet**

- Simple implementation using Python
 - Strong AES encryption standard
 - Cross-platform and open-source
- File Encryption Using Fernet (Data Security Project)

1. Secure File Storage

Encrypt confidential files (e.g., financial records, medical reports, personal documents) before saving them locally or in the cloud to prevent unauthorized access.

Example: Encrypting a personal diary or tax document before uploading to Google Drive or Dropbox.

2. Secure File Sharing

Share sensitive files over email or cloud platforms without worrying about interception—just share the encrypted file and a separate channel for the key.

Example: Sending encrypted project reports or confidential client data via email.

3. Data Privacy in Applications

Integrate encryption in software to protect user data such as login credentials, notes, or backup files.

Example: A note-taking app that encrypts user notes before saving them on disk.

4. Compliance with Data Protection Laws

Encrypting data helps meet regulations like GDPR, HIPAA, and others that require protection of personally identifiable information (PII).

Example: A healthcare app storing encrypted patient records to comply with HIPAA.

5. Educational Purposes

Use it as a project in schools, colleges, or coding bootcamps to demonstrate real-world use of cryptography and Python in cybersecurity.

Example: A mini-project for students learning about data security and Python.

6. Backup Protection

Encrypt backup files to prevent data leaks in case the backup drive or USB is lost or stolen.

Example: Encrypting backup copies of your website database or WordPress config file.

7. Password-Protected Archives

Build tools where files are encrypted and decrypted with a password or key, replacing traditional ZIP passwords with stronger encryption.

Example: A file encryption GUI tool that replaces WinRAR/ZIP encryption with Fernet.

8. Securing Logs and Reports

Encrypt server logs, exported database dumps, or application logs that contain sensitive data.

Example: Encrypting user logs before storing them on a production server.

Summary Table:

Use Case	Purpose
----------	---------

1. File Storage : Prevent unauthorized
access to saved files
2. File Sharing : Share files securely
across networks
3. App Integration : Add data privacy to
software
applications
4. Legal Compliance : Meet regulations
like GDPR, HIPAA
5. Learning & Education : Teaching
encryption
concepts in Python

6.Secure Backups : Protect data on
USB drives and external devices

7.Encrypted Archives : Replace weak
ZIP/RAR encryption

8.Log & Report Protection : Secure log
files or exported
reports

➤ **Limitations**

- Not suitable for extremely large files without modification
- Key must be stored and managed securely

- Not intended for enterprise-grade security systems without additional layers

➤ **Conclusion**

- Implemented a working file encryption-decryption system using Python
- Demonstrated real-world application of symmetric cryptography
- Simple, effective, and secure for local file security
- Can be further extended to GUI or cloud-based solutions

➤ **Future Scope**

- Add graphical user interface (GUI)
- Integrate with cloud storage APIs
- Add user authentication and logging
- Expand to mobile or web applications

➤ **References**

1. Python cryptography

documentation: <https://cryptography.io>

2. Fernet specification (symmetric encryption)
3. VTU Notes and Research Papers
4. Stack Overflow, GitHub, and online tutorials

Each member contribution:

1. Here's a contribution breakdown of the all 3 members involved in the project "Data Security and Encryption using Python (Fernet Algorithm)":

Each Member's Contribution is :

1. Jayshree Kumbar (2BA24CS046)

Researched Fernet algorithm and symmetric encryption methods.

Implemented the core functions:
generate_key(), encrypt_file(), and
decrypt_file()

Handled the Python coding and
debugging of the encryption/decryption
script.

2.Bhagya Akki (2BA24CS028)

- Designed the project workflow and user interface flow (command-line interaction).
- Collected and documented output screenshots (key generation, file encryption/decryption).

- Prepared the summary table of use cases and listed advantages/limitations of the project.

3.Kavita Rathod (2BA24CS055)

- Drafted the project report including the introduction, objectives, conclusion, and future scope.
- Explained the real-life applications of the encryption project (e.g., secure file storage, compliance).
- Helped in preparing the final documentation and presentation slides.
- Let me know if you'd like this formatted into a PDF or slide for your submission!

Overall Contribution (Team Work Summary)

- All three team members actively participated in the successful completion of the project. They collaborated on researching data security concepts, writing and testing Python code using the Fernet algorithm, designing the project workflow, collecting output results, and preparing the final documentation. Each member contributed equally to ensure the project was well-executed, technically sound, and clearly presented.